PROJET 01 — BIBLIOTHÈQUE

Guide complet de réalisation • Java 17+ • Programmation orientée objet

Dossier pédagogique basé sur les fichiers Livre.java, Utilisateur.java, Emprunt.java, Bibliotheque.java, Main.java et TestBibliotheque.java de la version finale fournie.

Objectif : comprendre le fonctionnement du projet, le reconstruire soi-même, le tester puis l'enrichir jusqu'à un niveau avancé.

Niveau de départ	Technologies	Mode de fonctionnement
Débutant / intermédiaire	Java 17+, JDK, terminal Windows, Git	Application console ; données conservées en mémoire

Sommaire : 1. Présentation • 2. Cahier des charges • 3. Architecture POO • 4. UML • 5. Terminal • 6. Concepts POO • 7. Réalisation • 8. Organisation • 9. Défis avancés • 10. Réussite.

1. Présentation du projet

L’application simule une petite bibliothèque. Un gestionnaire ajoute des livres, inscrit des utilisateurs, enregistre les emprunts et les retours, puis consulte les disponibilités et l’historique. L’interface est un menu texte interactif.

• Public cible : apprenant Java souhaitant pratiquer objets, collections, validations et logique métier.

• Périmètre actuel : une seule bibliothèque locale, aucune connexion réseau, aucune base de données.

• Livrable : code source compilable avec le JDK, tests simples et documentation.

2. Cahier des charges

Acteurs : opérateur de la bibliothèque (utilisateur du terminal) et lecteur inscrit (entité métier).

ID	Exigence fonctionnelle	Règle métier / résultat
F01	Ajouter un livre	ISBN, titre et auteur obligatoires ; ISBN unique.
F02	Rechercher un livre	Recherche par titre ou auteur ; recherche exacte par ISBN via le service.
F03	Inscrire un lecteur	ID strictement positif, nom non vide et ID unique.
F04	Emprunter	Le livre et le lecteur existent ; livre disponible.
F05	Retourner	Un emprunt actif doit correspondre à l’ISBN.
F06	Consulter	Listes de livres, lecteurs et historique des emprunts.
F07	Gérer les erreurs	Afficher un message en cas de règle violée.

Exigences non fonctionnelles : Java 17+, sans dépendances externes ; code lisible ; champs privés ; listes non modifiables depuis l’extérieur ; exécution dans le terminal ; validation des données.

Limites connues : les données disparaissent à la fermeture ; il n’existe ni date d’échéance, ni authentification, ni gestion de plusieurs exemplaires. La saisie d’un entier non valide peut interrompre le programme, car NumberFormatException n’est pas interceptée dans Main.

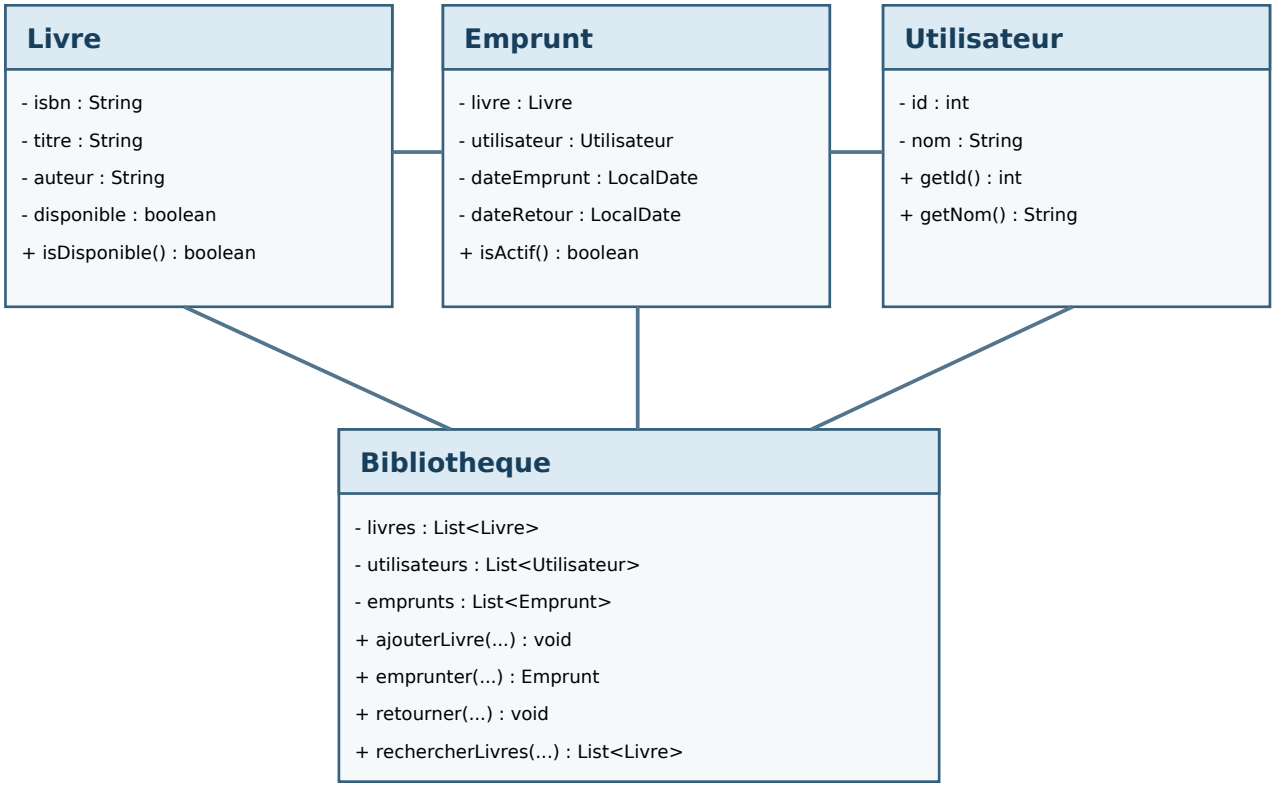
3. Architecture orientée objet

Classe	Responsabilité	Données principales
Livre	Représenter un ouvrage et sa disponibilité	isbn, titre, auteur, disponible
Utilisateur	Représenter un lecteur inscrit	id, nom
Emprunt	Relier un lecteur et un livre ; dater les mouvements	livre, utilisateur, dateEmprunt, dateRetour
Bibliotheque	Centraliser les collections et les règles métier	livres, utilisateurs, emprunts
Main	Menu, saisie et affichage	Bibliotheque, Scanner
TestBibliotheque	Vérifier les scénarios fondamentaux	assertions Java

Le découpage sépare les entités métier (Livre, Utilisateur, Emprunt), le service métier (Bibliotheque) et l'interface console (Main). Il n'utilise pas d'héritage dans sa version actuelle.

4. Diagramme UML des classes

Diagramme simplifié fidèle au code existant. Les signes « - » et « + » indiquent respectivement visibilité privée et publique ; les méthodes sans signe explicite dans le code peuvent avoir une visibilité de package.



Relations : Bibliotheque agrège des livres, des utilisateurs et des emprunts. Chaque Emprunt référence exactement un Livre et un Utilisateur. Main pilote Bibliotheque ; TestBibliotheque exerce ses méthodes.

5. Exemple d’exécution dans le terminal Windows

Dans PowerShell ou le terminal intégré de VS Code, se placer dans le dossier contenant src.

```
java -version
javac -version
javac -encoding UTF-8 -d out src\*.java
java -cp out Main
```

Exemple illustratif :

```
=== BIBLIOTHÈQUE ===
1. Livres
2. Ajouter un livre
3. Rechercher
4. Inscrire un utilisateur
5. Emprunter
6. Retourner
7. Emprunts
8. Utilisateurs
0. Quitter
Choix : 5
ISBN : 9780134685991
ID utilisateur : 1
Emprunt créé : Alice -> Effective Java (...)
Choix : 6
ISBN : 9780134685991
Livre retourné.
```

Les livres « Effective Java » et « Clean Code » ainsi que le lecteur Alice (ID 1) sont chargés au démarrage. Les dates affichées dépendent du jour d’exécution.

Tests : exécuter **java -ea -cp out TestBibliotheque** ; l’option -ea active les assertions Java.

6. Concepts POO à appliquer

• Classes et objets : Livre décrit une structure ; new Livre(...) crée une instance.

• Encapsulation : attributs privés, accesseurs publics et modifications contrôlées de la disponibilité.

• Constructeurs et invariants : refuser ISBN vide, nom vide et identifiant non positif.

• Composition / associations : Emprunt contient des références vers Livre et Utilisateur ; Bibliotheque gère leurs collections.

• Abstraction : Main appelle emprunter(...) sans connaître tous les détails des règles.

• Exceptions : IllegalArgumentException pour une entrée invalide ; IllegalStateException pour une opération impossible.

• Collections et généricité : List, List, List ; listes exposées en lecture seule.

• Polymorphisme, héritage, interfaces : non utilisés dans le code actuel ; à introduire dans les défis avancés plutôt que de les présenter comme déjà implémentés.

7. Étapes de réalisation

Étape	Travail à réaliser	Vérification
1	Créer le projet et le point d’entrée Main	Le menu s’affiche.
2	Implémenter Livre et Utilisateur	Constructeurs valident les entrées.
3	Implémenter Bibliotheque et les listes	Ajout et recherche fonctionnent.
4	Implémenter Emprunt	Dates et état actif cohérents.
5	Ajouter emprunt et retour	Disponibilité synchronisée.
6	Ajouter menu et gestion des erreurs	Scénarios exécutables au clavier.
7	Écrire les tests	Tests exécutés avec -ea.
8	Nettoyer, documenter et versionner	README, commits, cas limites.

Suggestion Git : une branche 01-bibliotheque, puis des commits atomiques comme « feat: ajouter gestion des livres », « feat: gérer emprunts », « test: couvrir retours ».

8. Organisation du projet

```
bibliotheque-version-finale/
|-- README.md
`-- src/
    |-- Livre.java
    |-- Utilisateur.java
    |-- Emprunt.java
    |-- Bibliotheque.java
    |-- Main.java
    `-- TestBibliotheque.java

(out/ est créé à la compilation)
```

Tous les fichiers Java sont actuellement dans le package par défaut. Pour professionnaliser l’application, créer src/main/java/fr/exemple/bibliotheque/ avec les sous-packages model, service, ui et exception ; ajouter src/test/java et un outil de build tel que Maven ou Gradle.

9. Défis supplémentaires : atteindre un niveau avancé

Niveau	Défi	Notions travaillées
1	Dates limites, pénalités de retard, plafond d'emprunts	LocalDate, Duration, règles métier
2	Gestion de plusieurs exemplaires d'un même titre	Modélisation, identité et stock
3	Sauvegarde JSON/CSV puis base SQLite ou PostgreSQL	Persistance, repository, JDBC
4	Interfaces Repository, service injectable	Abstraction, inversion des dépendances
5	Types de documents Livre, Magazine, DVD	Héritage, polymorphisme, classes abstraites
6	Rôles bibliothécaire / lecteur et authentification	Sécurité, séparation des responsabilités
7	Tests JUnit 5, cas limites, couverture	Tests unitaires et intégration continue
8	API REST Spring Boot, pagination, validation	Architecture multicouche, HTTP
9	Gestion concurrente de deux emprunts simultanés	Transactions, cohérence, concurrence
10	Pipeline GitHub Actions, logs et Docker	Qualité, livraison et déploiement

Parcours recommandé : finir les niveaux 1 à 3 avant de passer à une API REST. Écrire des tests de non-régression à chaque nouvelle fonctionnalité.

10. Critères de réussite

Critère	Condition de validation
Compilation	javac compile sans erreur sous Java 17+.
Fonctionnalités	Ajout, recherche, inscription, emprunt, retour et historique opérationnels.
Règles métier	ISBN et ID uniques ; double emprunt refusé ; retour d'un livre non emprunté refusé.
Qualité POO	Responsabilités distinctes, attributs privés, méthodes métier claires.
Robustesse	Entrées invalides traitées ; tester spécialement la saisie numérique du menu.
Tests	Assertions activées ; scénarios nominaux et erreurs couverts.
Documentation	README, diagramme UML, instructions Windows et exemples.
Versionnement	Commits compréhensibles ; branche dédiée et dépôt GitHub propre.
Niveau avancé	Au moins trois défis avancés réalisés avec tests et documentation.

Checklist finale : ☐ Compiler ☐ Lancer ☐ Ajouter ☐ Rechercher ☐ Inscrire ☐ Emprunter ☐ Bloquer double emprunt ☐ Retourner ☐ Vérifier historique ☐ Tester les erreurs ☐ Documenter ☐ Committer.

Note : ce PDF est un cahier de projet et un guide de progression ; il ne remplace pas le code source présent dans le ZIP.